



UNLOX

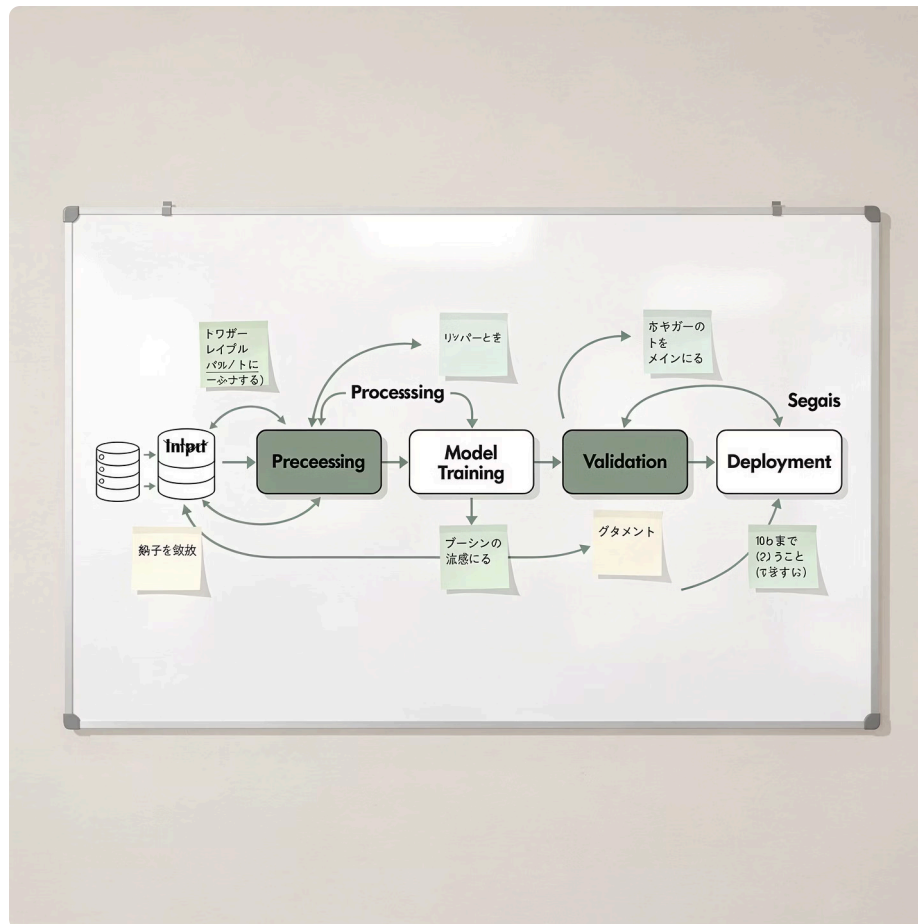
Managing the ML Lifecycle with MLflow

From experiment tracking to model deployment — a comprehensive guide to taming the chaos of machine learning development.

What Is MLflow?

MLflow is an open-source platform designed to manage the **end-to-end machine learning lifecycle**. It was created to solve one of the most persistent challenges in AI development: the chaos that arises when teams attempt to track hundreds — or thousands — of experiments without a unified system.

Without a structured approach, data scientists lose track of which hyperparameters produced the best results, which datasets were used, and which code version generated a specific model. MLflow brings order to this complexity by providing a cohesive set of tools that work together across every phase of model development.





UNLOCK

CORE COMPONENTS

MLflow's Three Pillars

MLflow is organized around three core components, each addressing a distinct phase of the ML workflow. Together, they create a seamless pipeline from experimentation to deployment.

Tracking

Log experiments and record parameters, metrics, and artifacts

Projects

Standardize code packaging for reproducible runs

Models

Package models in a universal format for any deployment target

MLflow Tracking — Your Experiment Logbook

What Gets Logged?

- **Parameters** — hyperparameters, learning rates, batch sizes
- **Metrics** — accuracy, loss, F1 score over time
- **Code Versions** — Git commit hashes for full traceability
- **Artifacts** — output files, serialized models, plots

MLflow Tracking is the foundation of the platform. It enables you to **log every experiment run** in a centralized store, creating a complete, reproducible history of your model training activities.

By recording all relevant metadata for each run, teams can easily **compare different models side by side**, identify the best-performing configurations, and trace any result back to the exact code, data, and parameters that produced it. This eliminates the "which notebook was that?" problem entirely.

UNLOCK

§ 26.1

MLflow Projects — Reproducibility by Design



Standardized Packaging

MLflow Projects define a convention for organizing your ML code — including dependencies, entry points, and runtime environment specifications — so that **any team member** can reproduce a training run with a single command.



Cross-Team Collaboration

By packaging code in a standard format (using Conda environments or Docker), teams eliminate the classic "it works on my machine" problem. Projects can be shared, versioned, and **run on any infrastructure** — from a local laptop to a cloud cluster.

MLflow Models — One Format, Any Target

The MLflow Models component provides a **standard packaging format** for machine learning models, regardless of the framework used to train them. Whether you built your model with scikit-learn, TensorFlow, PyTorch, or XGBoost, MLflow wraps it in a unified interface that simplifies downstream deployment.

1

Train

Build your model using any ML framework of choice

2

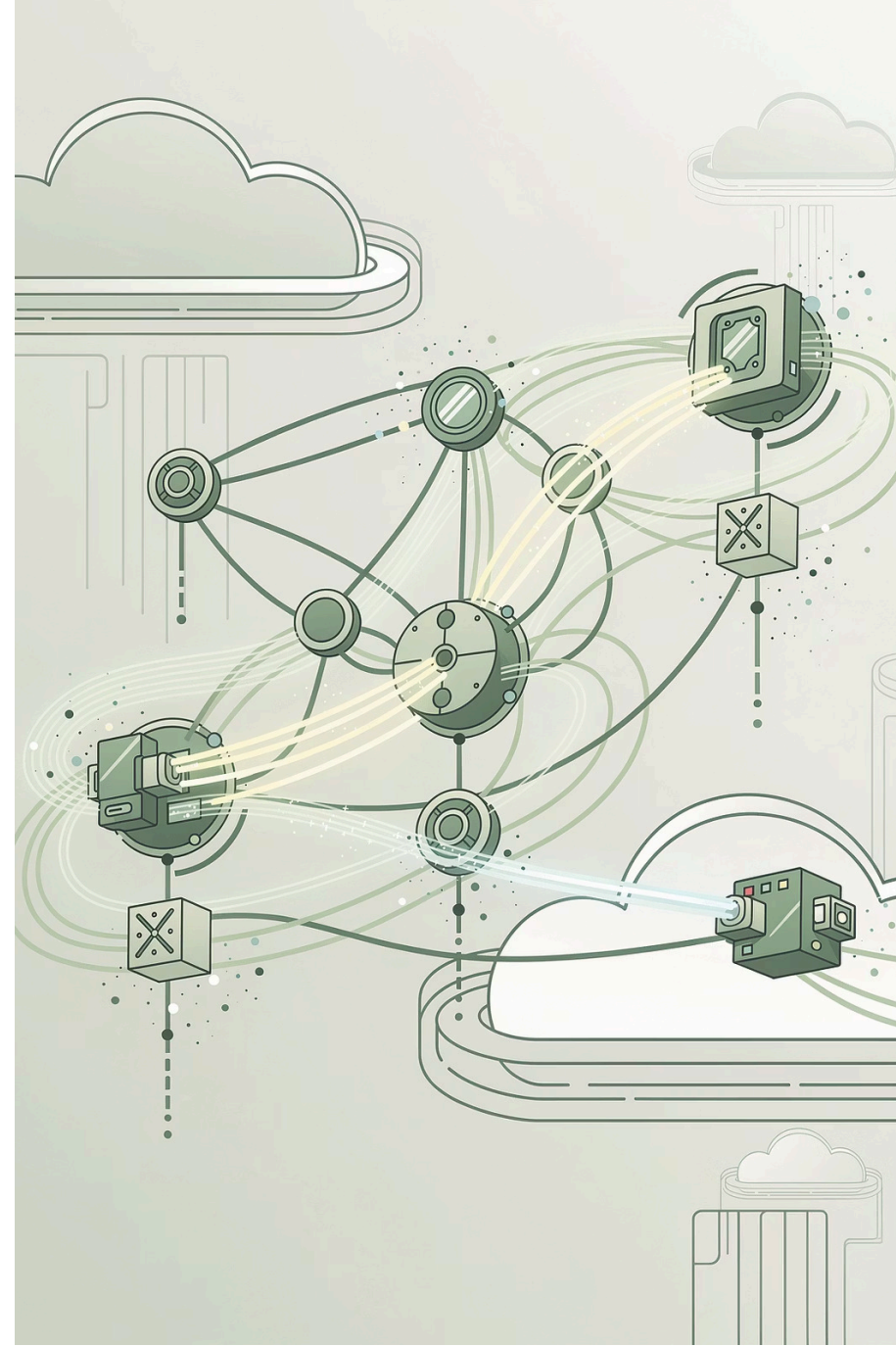
Package

Save as an MLflow Model with flavor-specific metadata

3

Deploy

Serve via REST API, batch inference, or edge devices





UNLOCK

CHAPTER 2

The Model Registry

Version control isn't just for code — your production models need it too.

Why a Model Registry Matters

The **Model Registry** is a centralized hub for managing your model's full lifecycle — from initial training through staging, production, and eventual retirement. It solves a critical governance challenge: **knowing exactly which model version is serving predictions at any given time.**

Without a registry, teams often rely on ad hoc naming conventions, shared drives, or tribal knowledge to track which model is deployed where. The Model Registry replaces this fragility with a structured, auditable system.



Version Control

Track every model iteration with full lineage



Rich Metadata

Store descriptions, tags, and source experiment links



Secure Artifacts

Centralized storage with access controls

UNLOCK

§ 26.2

Model Lifecycle Stages

The Model Registry organizes every registered model into distinct lifecycle stages. Teams can **transition models between stages** with full audit trails, ensuring governance and traceability at every step.



Development

Experimental models under active iteration and evaluation



Staging

Candidate models undergoing validation, A/B testing, and review



Production

Approved models actively serving live predictions



Archived

Retired models preserved for compliance and rollback

Rollback with Confidence



One of the most critical capabilities the Model Registry provides is **safe, instant rollback**. If a newly deployed model begins producing degraded predictions or unexpected behavior, teams can revert to a previous version in seconds — not hours.

- ❏ **Best Practice:** Always maintain at least one validated previous version in a "Production-ready" state so rollback requires no additional validation steps.

Combined with version-controlled artifacts and full metadata logging, this ensures your production ML systems are as robust and recoverable as any traditional software deployment.

The Complete MLflow Workflow



MLflow unifies the entire ML lifecycle into a single, cohesive platform. By combining experiment tracking, reproducible project packaging, standardized model formats, and a governed Model Registry, teams move from chaotic ad hoc workflows to a **structured, auditable, and scalable** machine learning practice.



UNLOCK

KEY TAKEAWAYS

What to Remember

Track Everything

Log parameters, metrics, code versions, and artifacts for every experiment run

Standardize Packaging

Use MLflow Projects and Models to ensure reproducibility across teams and platforms

Govern with the Registry

Version your models, manage lifecycle stages, and always be ready to roll back safely